ZK 专家养成课程 · 基础篇（S01–S06）

> 本卷由 6 讲合并装订而成（内容与单文件版逐字一致）。篇内各讲以 `# S/L/M xx` 标题分界，总纲见 `README_课程总纲.md`。

本卷目录

- **S01** · S01 · 从一个游戏开始：为什么需要证明系统
- **S02** · S02 · 钟表算术：有限域
- **S03** · S03 · 把整张表变成一个多项式（MLE）
- **S04** · S04 · 用探针审问整张表：EQ 与"求值 = 求和"
- **S05** · S05 · 随机挑战：作弊者的困境与 Fiat–Shamir
- **S06** · S06 · sumcheck：把 100 万个数的总和，压成一条探针

S01 · 从一个游戏开始：为什么需要证明系统

> 这一讲不出现任何术语。你只需要会加法。读完你将明白这门课要造的机器长什么样、为什么非造不可。

---

1. 游戏规则

假设你雇了一台服务器帮你做事，事情简单到可笑：

> 服务器拿到一张 **4 个数**的表 `[3, 1, 2, 0]`，任务是把它们加起来。

服务器干完活，回来告诉你：**"总和是 6。"**

你现在有两个选择，都不好：

- **选择 A：相信它。** 服务器可能有 bug，也可能被黑客改过程序，甚至可能偷懒根本没算。你没有任何核查手段。
- **选择 B：自己重算。** 表只有 4 个数，重算没问题。但把表放大到 **100 万个数**呢？重算一遍的时间和服务器自己算一遍一样多——你雇它就没意义了。

问题：有没有第三条路——我不用重算，但作弊一定会被抓住？

2. 第三条路长什么样

这条路要靠服务器在交回答案时，**多交一样东西**。我们暂且叫它"凭据"。凭据必须同时满足三个要求，一个都不能少：

要求	大白话	反面情形
短	凭据本身很小，你核对凭据的功夫远小于重算	凭据就是把 100 万个数再抄一遍——等于没省
不冤枉好人	服务器没作弊，凭据核对一定通过	系统动不动报假警，没人敢用
不放过坏人	服务器在任何一个环节作假，核对都会以极高的概率失败	作弊成本为零，凭据是摆设

第三条里"极高的概率"值得停下来想清楚：**我们追求的不是 100% 抓住作弊，而是把漏网概率压到一个物理上无意义的数字**——比如 $1/2^{128}$。$2^{128} \approx 3.4\times10^{38}$。打个比方：就算作弊者每秒重试 10 亿次、从宇宙大爆炸 trial 到今天（约 4×10^{17} 秒），也才试了 4×10^{26} 次，距离 3.4×10^{38} 还差着 12 个数量级。**"概率性安全"不是妥协，它是这类系统的标准安全口径**——密码学里没有 100%，只有"攻击代价超过宇宙资源"。

3. 两个角色

从现在起，故事里有两个人：

- **证明方 P**（prover）：干活的、给出答案和凭据的服务器；
- **验证方 V**（verifier）：雇人的、只想花小力气确认答案的你。

记住这两个字母就够了。后文所有协议都是 P 和 V 之间的对话或"对话的代替品"。

4. 为什么这个游戏不平凡（先自己想 3 分钟）

回到 4 个数的表。请你现在放下文档，想一想：**如果服务器只告诉你"总和 = 6"和某个很短的凭据，你凭什么能抓住作弊？**

大多数人最先想到"抽查"：随机抽表里几个数自己加加看。但它抓不住"总数对、其中两数互相补偿"的作弊（比如把 1 改成 4、把 2 改成 −1，总和不变）。也想到"让服务器把表发来"——那就是选择 B。

想不出很正常。这个问题的答案需要一件反直觉的武器：**把整张表看成一个多项式，然后用"随机的一个点"去审问它**。一个点怎么能代表整张表？因为——表和多项式之间存在一种严格的一一对应，而"两个不同的表"对应的多项式"几乎在所有点上都不同"。这就是接下来三讲（S02–S04）要一砖一瓦建起来的东西。

5. 本课程的路线（先把地图给你）

讲	你将亲手做出的东西	它在整台机器里的角色
S02	一个"钟表世界"：加减乘除都封顶、循环的数系	所有计算发生的场地
S03	把任意长的表，变成一个可以"在任意位置取值"的多项式	数据的统一形状
S04	用一根"探针"审问整张表：一个点上的取值 = 全表的一次加权和	从"查表"到"求和"的关键一步
S05	随机挑战：为什么作弊者最怕验证方随机撒点	安全性的来源
S06	**sumcheck**：把"100 万个数的总和"压缩成 60 个数的凭据	发动机本体
提高篇	承诺（表没被调包）、查表（非线性函数）、整套系统	发动机装上整车

到 S06 结束，你将能**亲手在纸上跑完一遍完整的证明协议**（包括作弊者的每一种挣扎和每一次被抓）——那时再回头看今天的 4 个数，你会看见一台完整的机器。

6. 动手作业（10 分钟）

1. 构造一个"总和正确但内容作弊"的具体例子：改表中的两个数，使总和仍是 6。验证抽查两个位置时恰好漏掉它的概率。
2. 把"漏网概率 $1/2^{128}$"换算成"每秒 10 亿次重试需要多少年"（$2^{128} \approx 3.4\times10^{38}$，一年 $\approx 3.15\times10^{7}$ 秒）。把结果写成一句话。
3. 思考题：如果服务器说"总和是 7"而真实是 6，你需要什么样的信息才能不重算全表就否定它？（不用答出来——这就是 S06 的内容。）

把答案写进 `homework/s01.md`。

S02 · 钟表算术：有限域

> 上一讲说要造一台"作弊必被抓"的机器。这台机器的所有零件都造在一个特殊的数的世界里。这一讲你亲手把这个世界搭起来——从 13 格的钟表开始，最后一步换到真实的 254 位大数。 读完后你能够：在"钟表世界"里熟练做加减乘除；能证明"为什么除法只在素数钟表里人人都有"；知道真实系统用哪个数、为什么。

---

1. 一个只有 13 个数的钟

想象一个钟，但表盘上不是 1–12，而是 **0 到 12**，一共 13 格。指针指向 11 时再走 2 格，不叫 13，叫 **3**——绕回去了。所有加法都在这个钟上进行：

```
11 + 2 = 13 → 绕回 → 3          记作  11 + 2 ≡ 3 (mod 13)
9 + 7 = 16 → 16 − 13 → 5        记作  9 + 7 ≡ 5 (mod 13)
```

"≡ 3 (mod 13)"读作"在钟表意义下等于 3"，意思是**除以 13 余 3**。减法同理：3 − 5 = −2，钟上没有负数，往前借一圈：−2 + 13 = 11。所以 3 − 5 ≡ 11 (mod 13)。

乘法也是绕圈：4 × 5 = 20，20 − 13 = 7，所以 4 × 5 ≡ 7 (mod 13)。你可以在纸上把 13×13 的乘法口诀表全列出来——这就是这个世界里的全部乘法，没有一个例外。

2. 除法：这个世界最妙的部分

钟表世界里"除以 7"是什么意思？**找一个数，乘 7 后绕回 1**。挨个试（0 到 12 共 13 个候选，纸笔就够）：

```
7×1=7    7×2=14≡1   ← 就是它！
```

所以 7 的"倒数"是 2。7 ÷ 7 这种废话之外，连 1 ÷ 7 都有意义：1 ÷ 7 = 2。**在钟表世界里，每个非零数都能当除数**——这个性质叫"非零元素都有逆元"，我们把这样的世界叫**域**（field）。

但这件事不是随便哪个钟都成立的。看一个 6 格的钟（mod 6），问：2 的倒数是谁？把 2 乘遍 0..5：

```
2×0=0    2×1=2    2×2=4    2×3=6≡0    2×4=8≡2    2×5=10≡4
```

乘积只有 {0, 2, 4} 三个值，**永远撞不到 1**。罪魁祸首：2×3 = 6，绕圈后变成 0——两个非零数相乘得零！这种"暗门"叫**零因子**。有零因子的世界里除法残废。

为什么 13 格的钟没有暗门、6 格的有？ 因为 13 是素数、6 不是。给一个你能自己复述的证明：

> 在素数 p 格的钟里，取任意非零 a，看 a 乘上 0,1,2,…,p−1 的 p 个乘积。若有两个乘积相同：a·i ≡ a·j，则 p 整除 a·(i−j)。p 是素数且 p 不整除 a（a 非 0 且小于 p），所以 p 必须整除 (i−j)，而 i、j 都在 0..p−1 之间，只能 i = j——矛盾。**所以 a 的 p 个乘积两两不同，p 个位置塞满整个钟，1 必然在其中——逆元必然存在**。

用 mod 6 复验一遍证明的哪一步失效：2×0 ≡ 2×3，需要 6 | 2·(0−3) = −6 √ 成立，i ≠ j 也行——因为 6 不是素数，它可以"分期付款"地整除 2×3。**素数性挡住的正是这种分期付款**。（这两段请手推一遍，这是本课程第一个真正的证明，5 分钟投入。）

3. 负数：往回走 = 往前绕

钟上没有负号，"−3"就是从 0 **倒退** 3 格 = 前进 10 格。所以 −3 ≡ 10 (mod 13)。验算一致性：−3 + 10 = 7，而 10 + 10 = 20 ≡ 7 √ 两种算法殊途同归。一般规则：**−x ≡ p − x**（x ≠ 0）。

4. 真实系统用的大钟

我们 13 格的钟用来学规则，真实系统用的是一台**有 2^{254} 格的大钟**。精确的数（来自本仓库依赖库的源码 `arkworks-algebra/curves/bn254/src/fields/fr.rs:4`，以后所有常数我们只认源码）：

```
r =
21888242871839275222246405745257275088548364400416034343698204186575808495617
```

- 它是素数，位数 254（因为 $2^{253} \approx 1.447\times10^{76} < r \approx 2.189\times10^{76} < 2^{254} \approx 2.895\times10^{76}$）；
- 所有规则与 13 格钟**完全相同**，只是口诀表大到没人能打印——这正是用计算机的原因；
- 常见错误预警：网上最容易搜到的"BN254 常数"十六进制 `0x30644E72…F0000001` 是另一个数（曲线坐标用的域），**不是**这个 r。两个域只差最后三分之一的数字。本课程所有算术都在 r 上。

为什么非要绕圈，不直接用普通整数？ 三个理由，每个都会在后面兑现：

1. **数字不膨胀**：普通乘法 2^{500} 位数会爆炸，钟表永远封顶在 254 位——计算机存得下、硬件算得动；
2. **除法人人有份**（第 2 节）——后面的协议要大量"除"；
3. **它是均匀的骰子**：S05 的"随机挑战"要在一个有限、均匀的数集里掷骰子，钟表天然就是。

5. 整数世界 → 钟表世界：域化

推理程序里的数据是普通整数（含负数），进证明机器前要搬进钟表：正数原样进，负数加一圈：**−3 → r − 3 = …95614**。搬完之后在钟里做的每一次加、乘，结果都与"整数世界算完再搬进来"一致——只要整数运算本身没有溢出（本仓库用 64 位整数跑推理，模型的量化设计保证不溢出，这也是为什么量化位宽在 zkML 里是件严肃的事）。

动手验证这个"一致性"（第一次把钟表用起来）：在 13 格钟里算 (−3) × 7：钟表做法 10 × 7 = 70 ≡ 70 − 65 = **5**；整数做法 −21，而 −21 + 26 = **5** √。

6. 代码对应（现在你已具备读懂它的资格）

- 这个"钟"在代码里叫 `Fr`（BN254 曲线的标量域），Rust 类型 `ark_bn254::Fr`；加、减、乘的钟表规则由 `ark-ff` 库实现；
- "域化"的代码：`to_field`（`zkml/src/quantization/`），核心就是第 5 节那条规则；
- 工程细节预警（现在知道即可，S04 后回看）：`Fr` 内部用"Montgomery 表示"存数——那只是让大钟乘法跑得快的存储技巧，**对外行为与本章的钟表逐位一致**，你的手算结果可以直接和代码输出对拍。

7. 动手作业（30 分钟，全部在 mod 13 里）

1. 算：7 + 9、4 × 5、3 − 8、5^{-1}（试出来）、1 ÷ 4（提示：4 × ? ≡ 1）。 答案核对：3、7、8、8（5×8=40≡1）、10（4×10=40≡1）。
2. 证明 mod 13 里 3 的逆元唯一。（提示：假设 a·3 ≡ b·3 ≡ 1，则 3(a−b) ≡ 0，用第 2 节的论证。）
3. 构造 mod 15 里的一个零因子，并找出一个没有逆元的数。
4. 写 Rust（10 行以内）：用 `ark-bn254` crate 验证作业 1 的全部答案。环境搭建见 `L07_代码精读路线.md` §0。

做完放入 `homework/s02/`。**作业 2 的证明要写全句子**——本课程从这一讲起训练"证明要经得起别人挑字眼"。

S03 · 把整张表变成一个多项式（MLE）

> S01 留下的问题：服务器给出"总和 = 6"，凭什么信？S02 给了场地（钟表世界）。这一讲解决第一个实质障碍：**表是一堆孤立的数，怎么把它变成一个"在任意位置都能取值"的函数？** 你会亲手把两张小表变成多项式、在"格子之间"的位置取值，然后才第一次听到"多线性扩展"这个名字。 前置：S02（mod 13 的加减乘、逆元）。全程 mod 13，所有数字可心算。

---

1. 先做一件小事：两个数的表

取表 `[7, 4]`。把它想成一个只吃 0 和 1 的函数：

```
f(0) = 7        f(1) = 4
```

问题：f(1/2) 等于几？——也就是"两个入口中间"的位置。最自然的答案：**把两点连成一条直线**。设 f(x) = a + b·x，代入两个已知点：a = 7，a + b = 4 ⟹ b = −3。所以：

```
f(x) = 7 − 3x
```

在钟表世界（mod 13）里求 f(1/2)。"1/2"是 2 的逆元：2 × ? ≡ 1 (mod 13)，试出 ? = 7（2×7 = 14 ≡ 1）。所以：

```
f(1/2) = f(7) = 7 − 21 = −14 ≡ 12 (mod 13)
```

验算直觉：12 是不是真的"在 7 和 4 正中间"？钟表里的"正中间"= (7+4) 的一半 = 11 × 7（1/2 = 7）= 77 ≡ 12 √。在 13 格的钟上，从 7 前进 5 格是 12，再前进 5 格是 4——**直觉完好地活了下来，只是"一半"变成了"乘 7"**。

记住这件事的形状：**2 个数 ↔ 一条直线；直线由 2 个数唯一确定；直线在任何位置的值都可以算**。"唯一确定"是核心——两张不同的表（比如 [7,5]）给出不同的直线（7−2x），它们只在各自的两点上"装作相同"。

2. 四个数的表：从直线到"二次元直线"

现在表有 4 个数：`[3, 1, 2, 0]`。排成一个 2×2 的方格（行、列各 0/1）：

```
            列 y=0    列 y=1
行 x=0        3         2
行 x=1        1         0
```

函数现在是两进两出的：f(行, 列)。模仿上一节——找"穿过多面体的最平的面"。要求 f 在每个变量上单独看都是一次的（这正是"多线性"三个字的含义，现在你还没有学这个词，先记住要求）。设四个"角落选择器"：

```
ℓ₀(t) = 1 − t    (t=0 时它 =1，t=1 时 =0——"选 0 行")
ℓ₁(t) = t        (t=1 时它 =1——"选 1 行")
```

则拼出：

```
f(x, y) = 3·ℓ₀(x)ℓ₀(y)  +  1·ℓ₁(x)ℓ₀(y)  +  2·ℓ₀(x)ℓ₁(y)  +  0·ℓ₁(x)ℓ₁(y)
```

为什么这个式子一定对：把 (x,y) 代入任何一个格子，比如 (1,0)：ℓ₁(1)=1、ℓ₀(0)=1，其余选择器全为 0，于是 f(1,0) = 1·1·1 = 1 √——每个格子自动取到自己的值。展开整理（自己算一遍）：

```
f(x, y) = 3(1−x)(1−y) + x(1−y) + 2(1−x)y
        = 3 − 3x − 3y + 3xy + x − xy + 2y − 2xy
```

```
= 3 − 2x − y + 0·xy
= 3 − 2x − y
```

验证四个角：f(0,0)=3 √ f(1,0)=1 √ f(0,1)=2 √ f(1,1)=0 √。而且结果出乎意料地简洁——连 xy 项都消掉了（这是我们运气好，一般表会留下 xy 项，见作业 1）。

3. 在"格子之间"取值

直线可以取 f(1/2)，方格呢？随便挑一个非格子点，比如 **(x, y) = (5, 4)**（mod 13 里这就是合法的"格子之间"）：

```
f(5, 4) = 3 − 2×5 − 4 = 3 − 14 = −11 ≡ 2 (mod 13)
```

这个 2 是一个**完全确定的数**——它由表 `[3,1,2,0]` 唯一决定，改表中任何一个数它都会变（下一讲会精确量化"怎么变"）。请你体感一下这件事的分量：

> **一张 2^n 个数的表，现在等价于一个函数；这个函数在"随机的一个点"上的取值，就像表的一枚指纹。** S05 会证明：作弊者想伪造指纹，几乎必然失败。

4. 放大到 100 万个数

规则完全一样，只是格子变成 n 维超立方：

- 表长必须补成 2 的幂：1,000,000 → 补零到 1,048,576 = 2^{20}，于是 f 有 **20 个变量**（每个变量取 0/1）；
- 展开式有 2^{20} 项（每项 = 表值 × 20 个选择器的乘积）——人算不动，但这正是给计算机准备的；
- **术语落点**：这个 f 叫**多线性扩展**（Multilinear Extension，MLE）。"多线性"= 对每个变量单独看都是一次的（第 2 节的要求）；"扩展"= 从 2^n 个点扩展到整个钟表世界的函数。名字现在才出现，而你已经会造它、会算它了。

代码对应（第一次，简短）：`into_mle()` 把张量按行主序拍平成 2^n 的表——表的**第 i 项就是 f 在"i 的二进制位"那个角上的值**。约定（全系统统一）：i 的第 0 位 ↔ 第一个变量。例：表 [3,1,2,0] 的下标 2 = 二进制 10 → 变量 (x=0, y=1) → 值 2 √ 与第 2 节的方格一致。

5. 动手作业

1. 对表 `[1, 0, 0, 1]`：写出 MLE 的展开式，验证四角，算 f(2, 3)（mod 13）。（答案：f = (1−x)(1−y) + xy = 1 − x − y + 2xy；f(2,3) = 1−2−3+12 = 8。）
2. 对表 `[7, 4]` 的直线：求 f(9)（mod 13），并用"钟表中点"直觉验证（f(9) 应等于 7 和 4 在钟上按 9/13:4/13 加权的值——直接用插值公式核对）。
3. 思考：两张表 [3,1,2,0] 与 [3,1,2,1] 的 MLE 差在哪一项？这个差在哪些 (x,y) 上等于 0？（先列式，答案下一讲用：差 = $-\ell_1(x)\ell_1(y) = -xy$，等于 0 ⟺ x=0 或 y=0。）
4. Rust（15 行内）：把作业 1 的 f 在 (2,3) 的手算结果与代码对拍（用 `ark_bn254::Fr` 或先直接用 `i64 % 13` 练手）。

做完放入 `homework/s03/`。

S04 · 用探针审问整张表：EQ 与"求值 = 求和"

> 上一讲末尾你说出了那枚"指纹"：f(5,4) = 2。但你是**代入展开式**算出来的——展开式有 2^n 项，100 万个数的表你代入不动。这一讲教你算同一个值的**第二种方法**：把探针点变成一组"权重"，然后**只做一次全表加权求和**。两种方法结果必然相同——而这个"求值 = 求和"的等式，就是把 S01 的总和问题接进证明机器的那根轴。
> 前置：S03。全程 mod 13，数字延续 S03 的表 [3,1,2,0]。

---

1. 换个角度：不展开，用"选择器"直接算

S03 造 f 用的是四个角落选择器的加权和：

$$f(x, y) = 3\cdot\ell_0(x)\ell_0(y) + 1\cdot\ell_1(x)\ell_0(y) + 2\cdot\ell_0(x)\ell_1(y) + 0\cdot\ell_1(x)\ell_1(y)$$

不要展开它（上一讲展开了，这一讲偏不）。直接把探针点 (5,4) 代进选择器：

$$\ell_0(5) = 1-5 = -4 \equiv 9 \qquad \ell_1(5) = 5$$
$$\ell_0(4) = 1-4 = -3 \equiv 10 \qquad \ell_1(4) = 4$$

于是：

```
f(5,4) = 3×[9×10] + 1×[5×10] + 2×[9×4] + 0×[5×4]
       = 3×90    + 1×50    + 2×36    + 0
       ≡ 3×12    + 1×11    + 2×10    + 0×7        (全部 mod 13)
       = 36 + 11 + 20 = 67 ≡ 2 (mod 13)
```

与 S03 代入展开式的结果 f(5,4) ≡ 2 完全一致 √——必然一致，因为它们是同一个函数的同一个值，只是算法不同。

看清这个算式的结构：**"在探针点求值"被改写成了"对表的每个格子，拿格子值乘一个权重，然后全加起来"**。四个权重恰好是四个"角落选择器在探针点的取值"：

```
w(格子) = ℓ(格子行)(5) × ℓ(格子列)(4)
w(0,0)=12   w(1,0)=11   w(0,1)=10   w(1,1)=7
```

2. 给权重起名字：eq

权重 w(格子) 的通用公式：对每个变量取 "1−探针值" 或 "探针值"，按格子坐标连乘。这个函数叫 **eq**（equality 的缩写，原因见下）：

```
eq(r, c) = ∏_每个变量 b    ( r_b · c_b + (1−r_b)(1−c_b) )
```

代 c = r 进去：每一项变成 r_b² + (1−r_b)² = r_b（钟表里二次项消掉，因为 r_b·r_b + (1−r_b)(1−r_b) = r_b + 1 − 2r_b + r_b² + 1 − 2r_b + r_b² … 直接代数字更快：r_b=5 时 c_b=5 → 5·1 + (1−5)·0 = 5 √）。所以 **eq(r, r) = 1**；而 c ≠ r 时至少一个坐标错开、那一项为 0 → **eq(r, c) = 0**（布尔格子 c 上）。**"只在探针点取 1、其余格子取 0"——这就是名字里 equality 的来历**：它是一个以多项式形式写出的"等号"。

于是核心恒等式（本课程最重要的一个等式，请抄在手边）：

```
┌──────────────────────────────────────────────┐
│  f(r)  =  Σ_{格子 c ∈ {0,1}ⁿ}  f(c) · eq(r, c)      │
│                                              │
│  在任意点求值   =   全表的一次加权求和                  │
└──────────────────────────────────────────────┘
```

为什么它是整门课的枢纽：S01 的问题是"证明一个**总和**"；S03 的机器只会"**求值**"。这个等式说二者是同一件事——**任何求值主张都能改写成一张表的加权求和**，而"证明求和"正是 S06 的 sumcheck 唯一会做的事。没有这个等式，sumcheck 与你的数据无关；有了它，sumcheck 能审问任何表。

3. 权重表怎么算：倍增法（这正是代码里的算法）

4 个权重可以手乘。1,048,576 个权重呢？逐个按公式算要 $2^{20} \times 20$ 次乘——太慢。实际算法是**倍增**：从 `[1]` 出发，每吸收一个坐标，表长翻倍：

```
处理 y=4：  [1] → [1−4, 4] = [−3, 4] ≡ [10, 4]
处理 x=5：  每个旧项 k 拆成两项：k×(1−5) 和 k×5
            [10,4] → [10×(−4), 10×5, 4×(−4), 4×5] = [−40, 50, −16, 20] ≡
[12, 11, 10, 7]
```

结果 [12, 11, 10, 7] 与第 1 节手乘的四个权重**完全一致** √。算法只需 2^n-1 次乘法（每次翻倍每项乘一次），比逐项快 20 倍（n=20 时）。

两个必查的性质（30 秒验证，终身受用）：

1. **权重之和恒等于 1**：12+11+10+7 = 40 ≡ 1 (mod 13) √。原因：把 f 换成全 1 的表，f(r) 显然 = 1（MLE 全 1 表 = 常函数 1），恒等式两端一比即得。
2. **索引位序约定**：上表的顺序是 [w(0,0), w(1,0), w(0,1), w(1,1)]——**下标第 0 位 ↔ 第一个坐标**（与 S03 的表约定一致）。倍增法里"后处理的坐标占低位"，所以真实代码按**从最后一个坐标处理到第一个**的顺序来，最终恰好得到这个排布。本仓库的约定（先已核实源码 `poly/eq.rs`）：**表下标第 b 位 ↔ 点 r 的第 b 个坐标**。

代码对应：倍增法 = `evals()`（dp-crypto `poly/eq.rs:48-70`）；恒等式右边的加权求和 = `evaluate_at_chi`（`poly/dense.rs:741`）。你现在已经能逐行读懂它们了——读的时候拿 (5,4) 的例子跟一遍。

4. 动手作业

1. 对 S03 作业 1 的表 [1,0,0,1]：用权重法算 f(2,3)（mod 13），要求：①列出四个权重 ②验证权重和 = 1 ③与上一讲代入法的结果 8 对账。 （答案：ℓ(2)=

[−1,2]≡[12,2]，ℓ(3)=[−2,3]≡[11,3]；权重 = [12×11, 2×11, 12×3, 2×3] = [132,22,36,6] ≡ [2,9,10,6]；Σ = 1×2+0×9+0×10+1×6 = 8 √；和 = 27 ≡ 1 √。）

2. 证明：eq(r,c) 的权重和在"c 取遍全部布尔格子"时恒等于 1（不许只验算——给论证。提示：全 1 表 + 第 2 节的恒等式）。
3. 用倍增法对点 r = (9, 2) 生成权重表（mod 13），并用它算表 [3,1,2,0] 的 f(9,2)，再与代入法 f(9,2) = 3−18−2 = −17 ≡ 9 对账。

做完放入 `homework/s04/`。**从这一讲起，"列出每一步的中间数字"是作业格式要求**——以后的协议证明里，"数字对不上"就是发现错误的唯一线索。

S05 · 随机挑战：作弊者的困境与 Fiat–Shamir

> S04 给了你审问工具："f(5,4) = 2" 这条主张可以改写成全表加权和。但有一个致命漏洞：**点 (5,4) 是谁选的？** 如果是证明方自己选的，他能挑一个对自己有利的点。这一讲解决"点必须不可预测"，并推出本课程的安全性核心——作弊者的困境。你将亲眼看到一个具体的作弊者怎么挣扎、有多少逃生机。 前置：S03、S04。全程 mod 13。

---

1. 证明方自选点 = 完全没有安全性

回到 S01 的游戏。表 [3,1,2,0]，真实总和 6。假设作弊服务器把表偷改成 [3,1,2,9]（最后一格 0→9，多加了 9），想声称"总和 = 15"。

如果由**服务器自己选**探针点：它随便挑一个点，伪造一张与真表在该点取值恰好相同的"假表"，或者干脆只发送对自己有利的数字。**自选考题的考试没有意义**。结论：**探针点必须由验证方决定，且不可预测**。

2. 亲眼看着作弊者挣扎

现在让**验证方**随机撒点。作弊者的表 g̃ = [3,1,2,9]，真表 f = [3,1,2,0]。两者的 MLE 差是多少？（S03 作业 3 你已经算过一半）：

```
g̃ − f 的表 = [0,0,0,9] → 差多项式 D = 9·ℓ₁(x)ℓ₁(y) = 9xy
```

作弊者要不被抓，必须让 V 探到的点上 f 与 g̃ 取值相同，即 **D(x,y) = 0**，即 xy ≡ 0 (mod 13)，即 **x = 0 或 y = 0**。

V 的两个坐标各从 {0,…,12} 均匀抽一个：总共 13×13 = 169 种等可能组合，其中"x=0 或 y=0"的有 13+13−1 = 25 种。**作弊者逃生的概率 = 25/169 ≈ 14.8%** ——在 13 格的小钟上，八成半会被抓。

现在把钟换成真实世界：坐标从 [0, 2^{128}) 均匀抽（S02 的大钟 + 128-bit 随机数，来源下一节讲）。"x=0 或 y=0"的概率变成：

$$1 - (1 - 2^{-128})^2 \approx 2^{-127}$$

同一个作弊、同一个数学，逃生概率从 14.8% 掉到 2^{-127}——每秒 10 亿次重试也要 10^{30} 年量级。这就是 S01 承诺的"1/2^{128}"第一次落到实处。

把刚才的过程抽象成一句话（这就是 Schwartz–Zippel 引理的实体）：两张不同的表 → 两个不同的 MLE → 差是一个**不恒为零**的多项式 D；D 的根（使 D=0 的点）是有限个；**随机探针撞进根集合的概率 = 根集合占整个点的世界的比例**——小钟上是 25/169，大钟上是约 2^{-127}。注意根的个数由 D 的次数控制：例子里 D = 9xy 在每个变量上一次 → 每个变量至多 1 个根 → 根占比 ≈ (每个变量根数/13) 的组合。**次数越低根越少、作弊越难**——这句话在 S06 会变成精确的健全性公式。

3. 验证方不可能一直在场：用哈希当公证人

"V 每次都来现场发随机点"太贵——真实系统里 P 是服务器、V 是网页/合约/审计员。替代方案：

> **把"对话的完整记录"喂进一个密码学哈希函数（Blake3），取哈希输出当随机挑战。**

哈希的性质：输入变一个字节，输出面目全非且**事先无法预测**。于是协议变成：

$$P \text{ 发消息 } m_1 \to \text{双方算 } H(\text{记录}\|m_1) \text{ 得挑战 } r_1 \to P \text{ 发 } m_2 \to H(\text{记录}\|m_1\|m_2) \text{ 得 } r_2 \to \dots$$

V 事后只需拿到全部消息，按同样顺序重算哈希、重做同样的检查——**不需要在场**。作弊者能不能"多掷几次骰子"？能——改一版消息哈一次，逃逸概率每次 ≈ 2^{-127}；试到宇宙热寂也试不完 2^{128} 次。**这就是随机性的全部来源，也解释了为什么它必须不可预测：一旦可预测，第 1 节的"自选考题"复活。**

术语落点（现在机制已懂，名字一句话）：交互版协议（P、V 现场对话）叫交互式证明；"用哈希代替 V 发挑战"的改造叫 **Fiat–Shamir 变换**；改造后的协议叫非交互式。文献里的 IOP（交互式预言机证明）还要再加一个零件——"P 对大数据做**承诺**、V 只查指定位置的值"——那个零件是提高篇 S07（多项式承诺）的主角，到时候你已有全部前置。

4. 工程生死线（本仓库的真实约束）

哈希当公证人有一个铁律：**P 和 V 喂给哈希的字节必须一个不差**。顺序不同、编码不同、漏一个字节 → 挑战全错 → 证明作废。在本仓库里：

- 负责"记录对话"的对象叫 **transcript**（Blake3 哈希器封装，`dp-crypto/arkyper/transcript/`）；
- 挑战 = 16 字节哈希输出解释成钟表数 → **挑战空间 [0, 2^{128})**（源码注释原文：*"for challenge we dont need 256 bits we only need 128 bits"*，transcript/mod.rs:29-34）——第 2 节用的正是这个口径；
- 每一处"往记录里写了什么"都登记在 FPGA 文档附录 A 的标签总表里——**你以后写任何证明代码，第一件事就是查那张表**。

5. 动手作业

1. 把第 2 节的作弊者换成"偷改第一格"：g̃ = [9,1,2,0]。求差多项式 D、根集合、逃生概率（mod 13 与 2^{128} 世界各算一遍）。（答案：D = 6ℓ₀(x)ℓ₀(y) = 6(1−x)(1−y)，根 = x=1 或 y=1，概率同样 25/169 ≈ 14.8%；2^{128} 世界 ≈ 2^{-127}。）
2. 作弊者把表改成 [3,1,2,9] 但声称的总和仍是 6（即 9 抵消了别处 −9，表 = [−6,1,2,9]）："总和"这个单一检查抓不到它——写出这张表在**探针点 (5,4)** 的值与真表的差（模 13），说明为什么"多点探针"比"只验总和"强。（答案：假表 MLE = f −9ℓ₀(x)ℓ₀(y)+9ℓ₁(x)ℓ₁(y)；在 (5,4)：差值 Δ = −9×12 + 9×7 = −108+63 = −45 ≡ 7，故 g̃(5,4) = 2+7 = 9 ≠ f(5,4) = 2——单点探针就把它暴露了。）
3. 思考：为什么挑战用 16 字节而不是 4 字节？用第 2 节的公式算出 4 字节（32 bit）世界里作弊者的逃生概率，并解释这个概率为什么不可接受。

做完放入 `homework/s05/`。

S06 · sumcheck：把 100 万个数的总和，

压成一条探针

> 前四讲的所有零件在这一讲合体。你将亲手跑完一个完整的证明协议：证明方怎么用"发直线"代替"发全表"，验证方怎么用三轮算术检查代替重算，作弊者在每一轮有多少逃生机、最终在哪里被抓。这个协议叫 **sumcheck**——名字在协议跑通之后才出现。 前置：S03（MLE）、S04（求值=求和）、S05（随机挑战）。全程 mod 13，表 [3,1,2,0] 贯穿到底。

---

1. 问题的最后一次重述

表 f 的 4 个格子值 [3,1,2,0]，证明方声称：**Σ f = 6**（四个格子全加）。验证方想不读全表就核实。

S04 的恒等式告诉我们"求值可以改写成加权求和"，但现在的目标是反方向——**"求和"这个主张本身怎么廉价证明？** 直接的想法：验证方随机抽点（S05）也只审到局部。需要新机关。

2. 机关：不要一次加完，一次只"削掉"一个变量

四个格子的总和按变量拆开写：

```
Σ = Σ_{x∈{0,1}} Σ_{y∈{0,1}} f(x, y)
```

证明方的第一条消息：把 y 先加掉，只留 x——把"剩下一个变量的函数"发给验证方。用 S03 的展开式 f(x,y) = 3−2x−y：

```
g₁(X) = f(X,0) + f(X,1) = (3−2X) + (2−2X) = 5 − 4X
```

注意 g_1 是一**条直线**（两个系数就能完全确定），证明方只需发 2 个数。发哪 2 个数？发直线在 0 和 1 的取值：**$[g_1(0), g_1(1)] = [5, 1]$**。（直线由 2 点唯一确定——S02 的插值思想。）

验证方的检查：这条直线在两端点的和必须等于声称的总和：

```
g₁(0) + g₁(1) = 5 + 1 = 6 ✓  ← 通过
```

这一步在逻辑上等于说："**我相信"Σ_y f(X, y) 确实是你发的这条线 g₁(X)""**"。可这条线可能是假的——验证方现在抽查它。

验证方的随机挑战（S05 的机制）：撒一个点，mod 13 里掷出 **r₁ = 7**。双方计算：

```
g₁(7) = 5 − 28 = −23 ≡ 3 (mod 13)
```

验证方用手头的 2 个点插值（他不需要知道直线的表达式）：g₁(7) = 5 + 7×(1−5) = 5 − 28 ≡ 3 ✓ 一致。

此时协议的状态发生了一次"变身"：原来的主张"总和 = 6"变成了新主张

```
Σ_{y∈{0,1}} f(7, y) = 3
```

——第一个变量已被随机数**钉死**，剩下的还是一个"某张表的某个总和"问题，但表小了一半。**如此往复，直到没有变量可削**。

3. 第二轮：表被折叠，主张被续写

新主张关心"f 在 x=7 处的切片"。双方都把表沿 x=7 折叠（用 S04 的权重思想：f(7,y) = (1−7)·f(0,y) + 7·f(1,y)，注意 mod 13 里 1−7 = −6 ≡ 7）：

```
折叠后的表：  y=0: 7×3 + 7×1 = 28 ≡ 2      y=1: 7×2 + 7×0 = 14 ≡ 1
            → 新表 [2, 1]
```

这张折叠表恰好就是 f(7, y) 的两个值——可以立刻验证：f(7,0) = 3−14−0 = −11 ≡ 2 ✓，f(7,1) = 3−14−1 = −12 ≡ 1 ✓。折叠不是近似，是**精确地固定了一个变量**。

第二轮与第一轮一模一样，只是主角换成折叠表 [2,1]（它作为 y 的函数是直线 h(Y) = 2 − Y）：

```
证明方发：[h(0), h(1)] = [2, 1]
验证方查：2 + 1 = 3 = g₁(7) ✓   ← 上一轮的尾巴接上了这一轮的头
```

```
验证方掷：r₂ = 4
最终主张：f(7, 4) = h(4) = 2 + 4×(1−2) = 2 − 4 = −2 ≡ 11 (mod 13)
```

直算对账：f(7,4) = 3 − 14 − 4 = −15 ≡ 11 √。**协议结束**。

4. 复盘：这台机器做了什么

整个凭据只有 4 个数字（两轮各 2 个）加 2 个随机挑战：

```
P 发：[5, 1]   →   V 掷 r₁=7   →   P 发：[2, 1]   →   V 掷 r₂=4
最终：V 持有主张 "f(7,4) = 11"
```

- **完备性**：每一步诚实的证明方都能通过（你刚跟着走了一遍）；
- **简明性**：4 个数的表没显出优势，但看放大——**2^{20} 个数的表只需 20 轮、每轮 2~3 个数，凭据约 60 个数**，验证方全程只做"两点插值 + 一次加法"级别的检查，20 次而已；
- **那个最终主张怎么办**：V 手里没有表，没法核对 f(7,4)=11。这正是 S01 三要求里"不放过坏人"的最后一道闸——**表必须在协议开始前被"承诺"**（一种加密指纹，提高篇 S07 的主题），最终主张靠指纹核销。sumcheck 自己只负责把"百万数的总和"**归约**成"一个点的取值"；归约+指纹，合起来才是完整的机器。这个分工现在记住即可。

5. 健全性：作弊者的每一条路都通向被抓

这是本讲最重要的一节。设作弊者想声称"总和 = 5"（真值 6）。逐轮推演他的全部可能：

情形 A：第一轮就撒谎。 他必须发一条满足 g̃(0)+g̃(1) = 5 的线。取 g̃ = [3, 2]（即 g̃(X) = 3−X）。真实的"削掉 y"的线是 $g_1(X) = 5-4X$。两条不同的直线**至多在 1 个点相等**：3−X = 5−4X ⟹ 3X = 2 ⟹ X = 2×9 = 18 ≡ **5** (mod 13)（3 的逆元是 9，S02 作业 1）。所以：

> **V 掷出的 r_1 只要不等于 5，作弊当场进入死亡螺旋**：他此后必须围绕"g̃(7) = 9"这个假值继续圆谎——发一张"总和为 9"的折叠表（比如 [4,5]），但他的真表折叠后是 [2,1]（总和 3）。每个后续数字都被真表锁定，缺口一路传导到最后的主张 $f(r_1,r_2)$，在承诺核对处**必然**对不上（假链的终点 ≠ 真表在承诺指纹下的取值）。逃生概率 = 恰好掷中 $r_1=5$ = **1/13**。

情形 B：第一轮诚实，后面某轮撒谎。同样的分析在每一轮独立成立——每轮逃生概率 ≤ 1/13（一般公式见第 6 节），且任何一轮没逃掉，后面全部锁死。

情形 C：全程诚实，只在最后一步改最终值。最终主张 $f(r_1,r_2)$ 直接触发承诺核对失败。

把 1/13 换成真实世界的数字：轮多项式的"根"至多 d 个（d = 每个变量的次数：单表线性 d=1；两张表相乘 d=2），挑战空间 2^{128}（S05），所以**每轮逃生概率 ≤ $d/2^{128} \approx 2^{-127}$**；20 轮合计 ≈ $20\times2^{-127} \approx 2^{-122}\cdot1.5$。**这就是"不放过坏人"的精确含义，也是整个证明系统安全性的全部来源**。（注意分母是 2^{128}——挑战的熵——不是域的大小 2^{254}，S05 的口径在这里兑现。）

6. 两张表相乘：协议一字不改，只是"线"变成"抛物线"

真实计算里 sumcheck 面对的不是一个表，而是**表的乘积**——比如矩阵乘的断言 Σ_k A(·,k)·B(k,·)。设 h 的表为 [1,0,0,1]（S03 作业 1），看乘积 g = f·h：

```
g 的表 = [3×1, 1×0, 2×0, 0×1] = [3, 0, 0, 0]      总和 = 3
g 的 MLE = 3ℓ₀(x)ℓ₀(y) = 3 − 3x − 3y + 3xy
第一轮：g₁(X) = g(X,0) + g(X,1) = (3−3X) + (3−3X−3+3X) = 3 − 3X
```

g_1 仍是直线（这次碰巧）。**一般地，d 张表的乘积在削变量后是 X 的 d 次多项式**——d=2 时一条抛物线需要 3 个点确定，所以证明方每轮发 **d+1 个数**，其中超出已知点数的用**插值补齐**（如直线补 g(2) = 2g(1) − g(0)；抛物线补 g(2) = 2g(1) − g(0) + (g(2)−2g(1)+g(0))… 实现里用预计算的插值系数表，dp-crypto `sumcheck/extrapolate.rs`）。**健全性公式的 d 就是这么进来的**：d 次多项式至多 d 个根 → 每轮逃生 ≤ $d/2^{128}$。

验证方检查的全貌（n 轮后的完整清单）：①每轮 g_i(0)+g_i(1) = 上一轮在 r_{i−1} 的插值；②最后一轮结束，双方对最终点 $(r_1,\ldots,r_n)$ 达成一条求值主张 → 交承诺核对。V 的全部工作量 = n 轮 × O(d) 次钟表运算。

7. 现在去读代码（你会发现每个字都认识）

你刚学的东西	代码
"发直线在 0..d+1 的取值"	`build_uni_poly`（dp-crypto `sumcheck/prover.rs:528-577`）——逐"相邻对"算乘积的 d+1 个系数
"把表沿 r 折叠"	`fix_var` / `fold`（`dense.rs:454`）：z'[i] = z[2i] + r·(z[2i+1]−z[2i])，与第 3 节手算的折叠一字不差
"掷挑战"	`transcript.append_and_sample(b"Internal round")`（每轮一次，S05 的哈希公证人）
"多张表的乘积"	虚拟多项式机制（`sumcheck/virtual_polys.rs`）——把 d 张表的表达式交给同一个 sumcheck
"20 轮的循环"	`IOPProverState::prove`（`sumcheck/prover.rs:116`）；多线程版把表按线程切片并行折叠（Phase 1/2，arXiv:2210.00264）
成本账单	n=20、二表乘积：全程读约 8.39M 项（≈268 MB）、乘约 3.15M 次（推导见本讲作业 4）

8. 动手作业（本讲分量最重，请留足 2 小时）

1. **换挑战完整重跑**：$r_1 = 3$、$r_2 = 9$，在 mod 13 里把第 2–3 节全程重算（每步列中间数）。答案自检：折叠表应为 [f(3,0), f(3,1)] = [(3−6), (3−6−1)] = [−3,−4] ≡ [10, 9]；总和检查 10+9 = 19 ≡ 6 √；终值 f(3,9) = 3−6−9 = −12 ≡ 1。
2. **作弊实验**：声称总和 = 5，第一轮发 [4,1]（和为 5）。求真线 $g_1 = 5−4X$ 与假线 4−3X 的交点（逃生点），算出逃生概率；然后掷 $r_1 = 11$，推演作弊者后续每一步，指出最终在哪被什么抓住。（交点：5−4X = 4−3X ⟹ X = 1 → 逃生概率 1/13。r_1=11 时假线值 4−33 = −29 ≡ 10，真值 Σ_y f(11,y) = 5−44 = −39 ≡ 0——缺口已现，任何后续圆谎终点都过不了承诺。）
3. **两张表相乘**：对 g = f·h（h = [1,0,0,1]）完成两轮 sumcheck（挑战自选），验证每轮检查通过、终值与直算一致。
4. **复杂度账单**：按第 7 节的口径，推导 n=20、二表乘积、max_degree=2 的"总读取项数 / 总写出项数 / 总乘法次数"三个数，并换算成 MB（32 字节/项）。做完与 L07 §0 的 measure 输出对拍。
5. **Rust**（P2 正式题，见 L08）：实现单表版 sumcheck 的 P/V 两侧，与本讲手算对拍；再做"作弊必被抓"的负例测试。

做完放入 `homework/s06/`。

本讲你获得了什么

- 一台完整的"求和归约机"：任意 2^n 个数的求和主张 → n 轮对话 → 一条最终探针主张，凭据 O(n·d) 个数；
- 健全性的精确来源（每轮 $\le d/2^{128}$）与它的每一个组成部分（MLE 的低次性、随机挑战的 128-bit 熵、承诺兜底）；
- 读懂 sumcheck 引擎代码的全部前置知识。

下一讲（提高篇 S07）：最终主张 f(r)=11 悬在空中——承诺方案（KZG/HyperKZG）负责把"表没被调包"这件事钉死。你将看到 S03 的"指纹"直觉如何变成真实的密码学。